

Bank Customer Churn Prediction

A machine learning project: predicting which customers are about to leave, and why

0.864 Cross-validated ROC-AUC (XGBoost)	71% Churners caught (recall, test set)	55% Flags that were correct (precision, test set)	2.7x Better than random outreach
--	---	--	---

Executive summary

A bank loses revenue every time a customer leaves, and it is far cheaper to keep a customer than to win a new one. This project builds a model that scores each customer's likelihood of leaving, so a retention team can contact the right people first. It uses a public dataset of 10,000 bank customers, of whom 20.4% left.

Key findings

- **XGBoost was the best of three models** (mean 5-fold ROC-AUC 0.864), ahead of Random Forest (0.854) and Logistic Regression (0.766). It won in every one of the five folds against Random Forest.
- **Accuracy is misleading here.** Random Forest has the highest accuracy (85.9%) but catches only 44% of churners. The model was therefore judged on recall, precision and ROC-AUC.
- **The decision threshold was set to 0.55** using training data only, assuming a retention team that can contact about one in four customers. On the untouched test set the model flagged 26.4% of customers and caught 70.8% of those who left, with 54.6% precision.
- **The main drivers of churn** are customer age (highest risk at 41 to 60), the number of products held (3 to 4 products is very high risk, 2 is the safest), and whether the customer is an active member. Customers in Germany churn at about twice the rate of France and Spain.
- **The model is packaged as a Streamlit web app** that scores one customer or a whole CSV file and shows the reasons behind each score.

1. Business problem and objective

Customer churn means a customer stops using the bank. Every lost customer takes future revenue with them, and replacing them costs more than keeping them. If the bank could tell in advance who is likely to leave, it could act early with an offer, a phone call, or a fix to whatever is causing the problem.

Objective. Build a classification model that outputs a churn probability for each customer, choose a decision rule that fits the retention team's capacity, and explain the predictions in plain language.

Success criteria. Because only about one customer in five churns, overall accuracy is not a useful measure (a model that always predicts "stays" would be about 80% accurate and useless). Success was defined as:

- **Recall** on churners: of the customers who really leave, how many does the model find?
- **Precision:** of the customers the model flags, how many really would have left (each false alarm wastes a retention call)?
- **ROC-AUC:** how well the model ranks risky customers above safe ones, independent of any cutoff.
- **Stability:** results must hold up under cross-validation and on data the model has never seen.

2. Data

The dataset is the public "Churn Modelling" file for a bank with customers in France, Germany and Spain. It has 10,000 rows and 14 columns, with no missing values, no duplicate rows and no duplicate customer IDs. The target column is **Exited** (1 = left, 0 = stayed). 2,037 customers (20.4%) left, so the classes are imbalanced.

Column	Type	Description
CreditScore, Age, Tenure, Balance, EstimatedSalary	Numeric	Customer profile and account values
NumOfProducts	Numeric (1 to 4)	Number of bank products held
Geography, Gender	Categorical	Country (3 values) and gender
HasCrCard, IsActiveMember	Binary	Has a credit card; is an active member
RowNumber, CustomerId, Surname	Identifier	Dropped: they carry no behavioural signal
Exited	Target	1 if the customer left the bank

The data was split 80/20 into a training set (8,000 customers) and a test set (2,000 customers, 407 of them churners), using stratification so both parts keep the same churn rate. The test set was kept untouched until the final evaluation.

3. Exploratory data analysis

Before any modelling, the data was explored with charts to understand who churns. The patterns below shaped the feature engineering and later matched the model's own explanations.

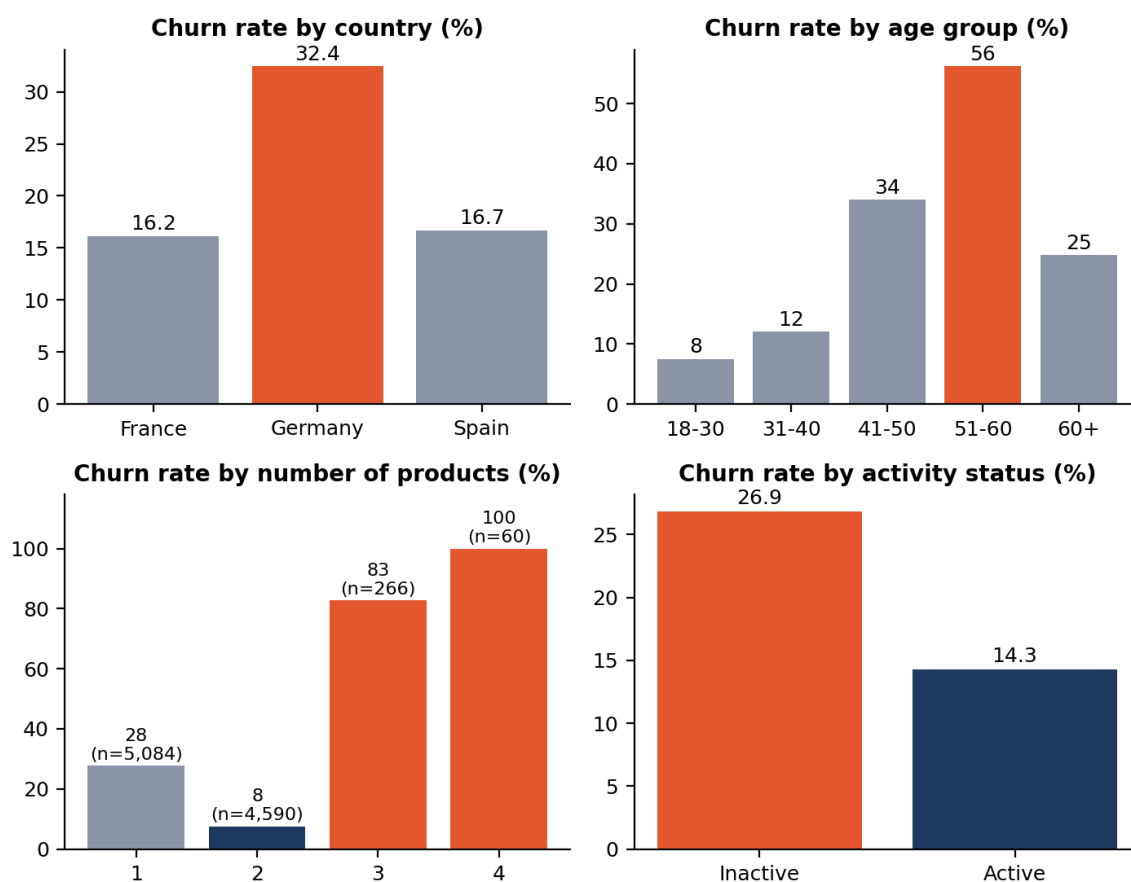


Figure 1. Churn rate by country, age group, number of products and activity status. Orange marks the highest-risk group in each panel.

Findings

- **Country.** Germany churns at 32.4%, about double France (16.2%) and Spain (16.7%).
- **Age.** Churn rises sharply through the 40s and peaks at **56.2%** for ages 51 to 60, then falls to 24.8% for customers over 60. The relationship is not a straight line.
- **Products.** Customers with 2 products are the most loyal (7.6% churn). Customers with 1 product churn at 27.7%, and those with 3 or 4 products churn at 82.7% and 100%. The 3 and 4 product groups are small (266 and 60 customers), so those rates should be read with care.
- **Activity.** Inactive members churn at 26.9% versus 14.3% for active members.
- **Gender.** Women churn more (25.1%) than men (16.5%).
- **Balance.** 36.2% of customers have a zero balance, and they churn *less* (13.8%) than customers with a balance (24.1%).

A lesson from the correlation matrix. Linear correlation with churn was weak for most features: Age +0.28, IsActiveMember -0.16, and NumOfProducts only -0.05. Yet the product chart shows number of products is one of the strongest signals. The relationship is a curve, which correlation cannot capture. This is why tree-based models were expected to do better than a linear one.

4. Feature engineering and preprocessing

All preprocessing lives inside a scikit-learn Pipeline, so it is fitted on training data only and applied identically to new customers. This prevents data leakage and guarantees that the deployed app transforms inputs exactly as in training.

Feature group	Features	Treatment
Numeric	CreditScore, Age, Tenure, Balance, EstimatedSalary, NumOfProducts	Standard scaling
Categorical	Geography, Gender	One-hot encoding (first category dropped)
Binary	HasCrCard, IsActiveMember	Kept as 0/1
Engineered	ZeroBalance (1 if Balance = 0)	New feature derived from the exploratory analysis

The target **Exited** was separated from the features (X and y) before splitting. After encoding, the model sees 12 inputs. Class imbalance was handled with class weights (Logistic Regression, Random Forest) and *scale_pos_weight* (XGBoost), which make the model pay more attention to churners without altering the data.

5. Model development and comparison

Three models of increasing flexibility were trained and compared: **Logistic Regression** (a simple, explainable baseline), **Random Forest** (many decision trees voting together) and **XGBoost** (trees built sequentially, each correcting the last one's errors). Each was evaluated first on the held-out test set at the default 0.5 cutoff.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.715	0.389	0.705	0.501	0.777
Random Forest	0.859	0.768	0.440	0.559	0.857
XGBoost	0.803	0.512	0.747	0.607	0.867

Table 1. Test-set results at the default threshold of 0.5 (2,000 customers, 407 churners).

Reading the table. Random Forest has the highest accuracy and precision but the lowest recall: it misses more than half of the churners, because it predicts "stays" for most customers and 80% of customers do stay. Judged by accuracy alone, the worst model for this job would have been chosen. XGBoost has the best recall, F1 and ROC-AUC.

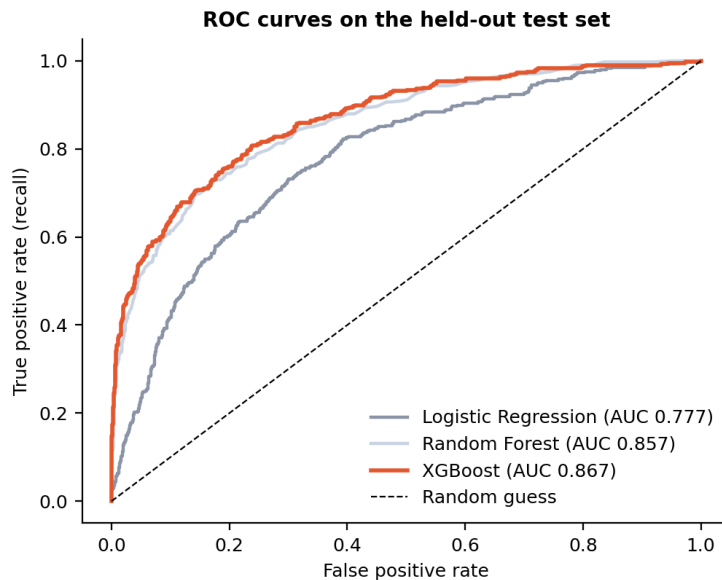


Figure 2. ROC curves on the test set. The closer a curve sits to the top-left corner, the better the model ranks risky customers.

Cross-validation

A single train/test split can flatter or hurt a model by luck, so the comparison was repeated with 5-fold stratified cross-validation on the training data. Each value below is the mean over the five folds, with the standard deviation in brackets.

Model	ROC-AUC	Recall	Precision	F1
Logistic Regression	0.766 (0.021)	0.684 (0.040)	0.380 (0.017)	0.488 (0.022)
Random Forest	0.854 (0.012)	0.441 (0.033)	0.775 (0.017)	0.562 (0.031)
XGBoost	0.864 (0.013)	0.732 (0.037)	0.522 (0.012)	0.609 (0.017)

Table 2. 5-fold cross-validation results (mean, with standard deviation in brackets).

- Logistic Regression is clearly the weakest (ROC-AUC 0.766). The gap to the tree models is far larger than the fold-to-fold variation.
- XGBoost leads Random Forest by only about 0.01 ROC-AUC, similar in size to the standard deviation. However, XGBoost scored higher in all five folds, so the lead is small but consistent.
- The recall gap is large: XGBoost 73% versus Random Forest 44%.
- Cross-validation scores are close to the test-set scores (XGBoost ROC-AUC 0.864 versus 0.867), so the test results were not a lucky split.

Selected model: XGBoost, for the best balance of recall, F1 and ROC-AUC, consistent across folds.

6. Choosing the decision threshold

The model outputs a probability, and the default rule "flag if the probability is at least 0.5" is rarely optimal. Lowering the threshold catches more churners but creates more false alarms; raising it does the opposite. The right cutoff depends on how many customers the retention team can actually contact.

To avoid tuning on the test set, the threshold was chosen using **out-of-fold predictions on the training data**: every training customer was scored by a model that had never seen them.

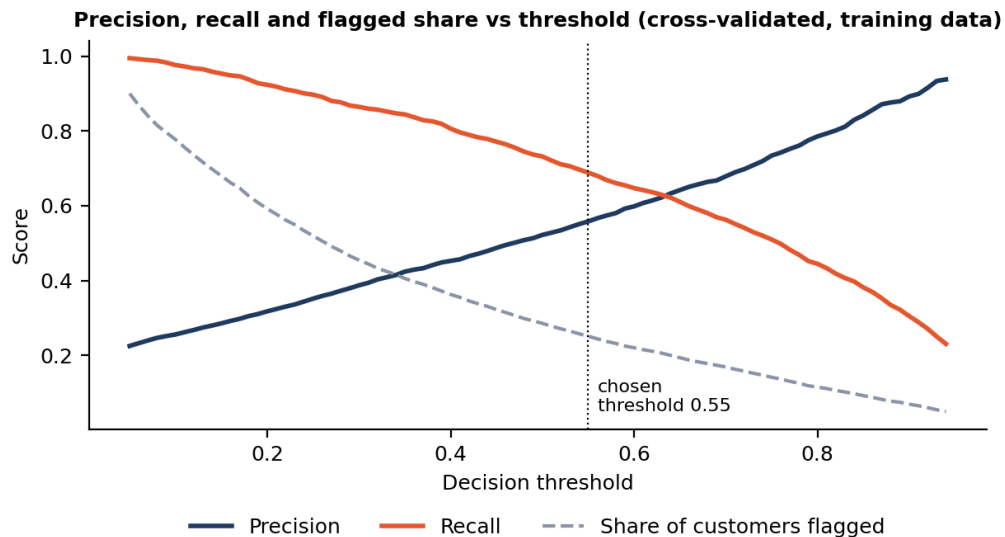


Figure 3. Precision, recall and the share of customers flagged as the threshold moves.

Threshold	Precision	Recall	F1	Customers flagged
0.30	0.387	0.864	0.535	45.5%
0.40	0.453	0.806	0.580	36.3%
0.50	0.522	0.732	0.609	28.6%
0.55	0.558	0.688	0.616	25.1%
0.60	0.598	0.647	0.622	22.0%
0.70	0.679	0.563	0.615	16.9%
0.80	0.785	0.445	0.568	11.5%

Table 3. Cross-validated trade-off between precision and recall (training data). The chosen threshold is highlighted.

Three ways of choosing the cutoff were compared

Strategy	Threshold	Outcome (training data)
Best F1 score	0.64	Best balance of precision and recall; flags about 20% of customers but leaves roughly 40% of churners unreached
Recall of at least 80%	0.40	Reaches 80% of churners with precision of about 0.46; flags about 36% of customers
Lowest cost (assumes a missed churner costs 10 times a false alarm)	0.26	Recall about 0.89, but flags about half of all customers; depends heavily on the assumed 10:1 cost ratio
Capacity-based (chosen)	0.55	Flags about 25% of customers (one in four), recall about 0.69, precision about 0.56

Why 0.55. The cost-based answer assumes every retention call saves the customer and uses a guessed cost ratio, and it would require contacting half of the customer base. A threshold sized to the team's realistic capacity (about one in four customers) is easier to defend, and the next step up in recall costs roughly twice as many calls per extra churner reached. In a real engagement, the cost ratio and capacity would come from the client.

7. Final test-set performance

With the threshold fixed at 0.55 from the training data, the model was scored once on the held-out test set of 2,000 customers.

Confusion matrix (test set, threshold 0.55)

	Predicted stay	Predicted leave
Actually stayed	1,354	239
Actually left	119	288

Metric	Result
Precision	0.546
Recall	0.708
F1 score	0.617
Customers flagged	26.4%
Churners caught / missed	288 / 119
False alarms	239

Figure 4 and table. Confusion matrix and headline metrics at threshold 0.55 on the test set.

What this means for the bank.

- Of 2,000 customers, the model flags **527**, and **288** of them would really have left.
- Contacting 26.4% of customers reaches **70.8%** of the customers who leave, about **2.7 times** better than contacting a random group of the same size (which would reach about 26%).
- The model misses 119 churners (29%) and raises 239 false alarms. Whether that trade is acceptable depends on the cost of a retention action versus the value of a retained customer.

Test results track the training-data estimates closely (recall 0.69 estimated, 0.71 observed; precision 0.56 versus 0.55), which indicates the model is not overfitting.

8. Explaining the model with SHAP

A churn score is only useful if the retention team understands it. SHAP breaks every prediction into the contribution of each feature: starting from the average customer, each feature pushes the score up (towards leaving) or down (towards staying).

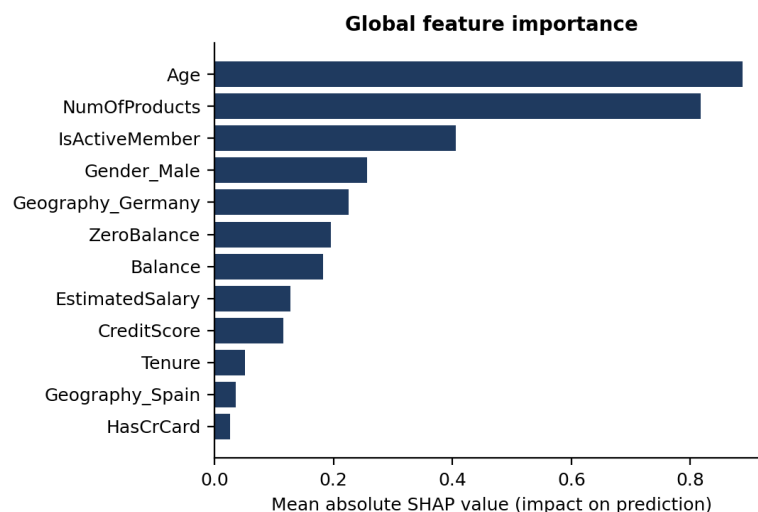


Figure 5. Average impact of each feature on the model's predictions (mean absolute SHAP value).

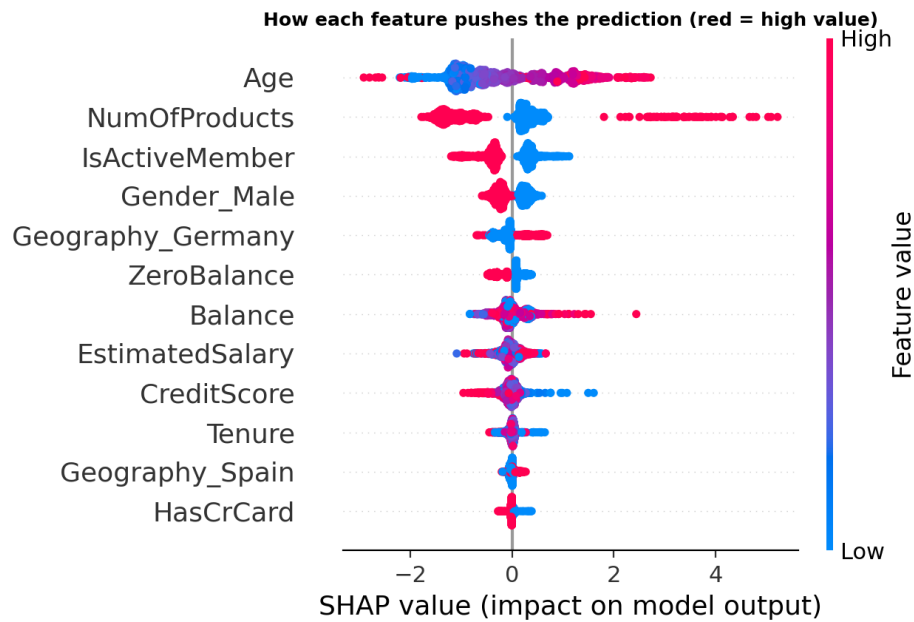


Figure 6. Each dot is one customer. Red means a high feature value; the horizontal position shows the push (right = towards churn).

Global findings

- **Age (0.89) and number of products (0.82)** are in a league of their own, followed by activity status (0.41), gender, Germany and zero balance.
- **Tenure, having a credit card and living in Spain** contribute almost nothing.
- The SHAP results agree with the exploratory analysis: the model relies on the same patterns found by hand, which increases trust in both.

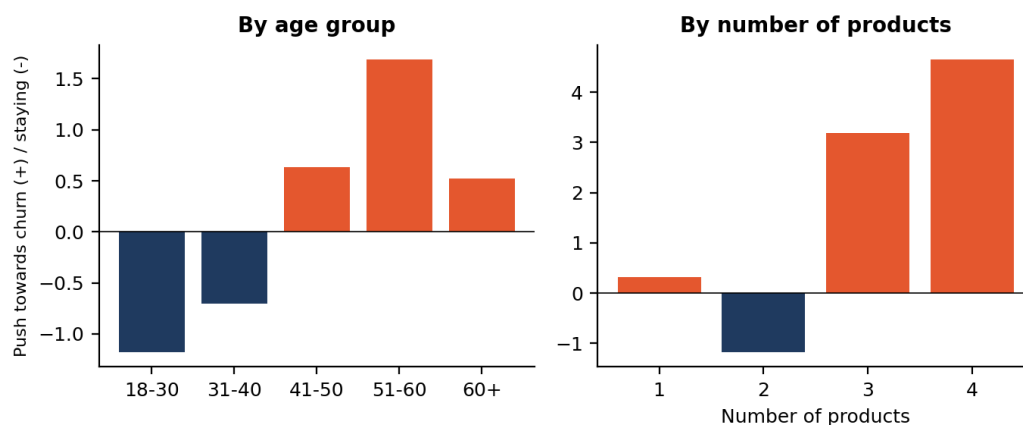


Figure 7. Average SHAP push by age group (left) and by number of products (right). Orange bars raise churn risk; navy bars lower it.

- **Age:** customers aged 41 to 60 are pushed towards churn (strongest at 51 to 60, +1.69), while customers under 40 are pushed away. The push is smaller for those over 60, matching the age chart in Section 3.
- **Products:** 3 and 4 products push very strongly towards churn (+3.18 and +4.65); exactly 2 products pushes away (-1.18).

Example: explaining individual customers

Highest-risk customer in the test set (churn score 99.95%): a 58-year-old man in Germany with 4 products, a balance of about 133,745, who is not an active member. The score is driven mainly by number of

products (+4.28) and age (+2.07), with smaller pushes from inactivity (+0.63) and balance (+0.46).

Lowest-risk customer (churn score 0.39%): an 18-year-old active member in Spain with 2 products, held back mainly by age and number of products.

Interpretation caution. SHAP shows what the model relies on, not what causes churn. For example, the model can show that customers with 3 to 4 products leave far more often, but only the bank can investigate whether fees, bundling or service quality is the reason.

9. Deployment: the churn prediction app

The trained pipeline (preprocessing and XGBoost) was saved with joblib and wrapped in a **Streamlit** web app so that non-technical staff can use it without touching code.

Feature	What it does
Single customer tab	Enter a customer's details and get a churn probability, a high-risk or lower-risk label, the top factors pushing towards leaving or staying, and a contribution chart
Batch CSV tab	Upload a CSV, get a probability and risk flag for every customer, sorted from highest to lowest risk, with summary counts and a download button
Threshold slider	Lets a manager change the cutoff live and see how many customers are flagged (default 0.55)
Input checks	Reports any missing columns in an uploaded file; the ZeroBalance feature is created automatically

Project files: `app.py` (the app), `churn_model.joblib` (the saved pipeline), `train_model.py` (retrains the model from the CSV), `requirements.txt` and `sample_customers.csv` (50 customers for demonstrations). The app can be run locally with `streamlit run app.py` and deployed to Streamlit Community Cloud from a GitHub repository.

10. Limitations and risks

- **Clean public dataset.** The data has no missing values or duplicates and may be partly synthetic. Real bank data is messier, and the approach would need adapting to a client's actual data.
- **One snapshot in time.** The data has no time dimension, so the model cannot be checked against how churn behaviour changes over months. Real churn projects need a clear churn definition and a prediction window.
- **Uncalibrated probabilities.** Class weighting inflates the scores, so they should be used for ranking and thresholding, not as literal probabilities. Calibration would be needed if exact probabilities matter.
- **Assumed costs.** The cost-based analysis used an assumed 10:1 ratio. The real value of a saved customer and the success rate of a retention offer must come from the business.
- **Small groups.** Customers with 3 or 4 products are only 326 of 10,000, so conclusions about them are less certain.
- **Prediction is not causation.** The model identifies who is at risk, not why they are unhappy, and does not show whether a given offer would keep them.
- **Fairness and compliance.** The model uses gender and country as inputs. In a real banking setting, use of such attributes needs a fairness and regulatory review, and might be removed.

- **Drift.** Customer behaviour changes (new competitors, price changes), so model performance will decline unless it is monitored and retrained.

11. Recommendations and next steps

- **Test the retention actions.** Run a controlled experiment where flagged customers receive an offer and a similar group does not, to measure whether the offers actually reduce churn and what they cost.
- **Investigate the high-risk segments:** customers with 3 to 4 products, customers in Germany, and inactive members aged 41 to 60.
- **Replace the assumed costs** with the bank's real numbers and re-select the threshold.
- **Tune and calibrate the model** (hyperparameter search, probability calibration) and compare against more feature engineering.
- **Monitor and retrain** on fresh data on a regular schedule (for example quarterly), and track recall and flagged share over time.
- **Review fairness** across gender and country before any real-world use.

Appendix: technical summary

Item	Detail
Language and libraries	Python; pandas, scikit-learn, XGBoost, SHAP, matplotlib, Streamlit, joblib
Split	80/20 stratified (random_state 42); 8,000 training and 2,000 test customers
Preprocessing	Standard scaling; one-hot encoding (drop first); ZeroBalance feature
XGBoost settings	300 trees, learning rate 0.05, max depth 4, scale_pos_weight = ratio of stayers to churners (about 3.9)
Random Forest settings	300 trees, balanced class weights
Logistic Regression settings	Balanced class weights, up to 1,000 iterations
Validation	5-fold stratified cross-validation on the training set; threshold from out-of-fold predictions
Final threshold	0.55 (about one in four customers flagged)